

개발 전 과정 보고서

Tiny Second-hand Shopping Platform

과목	시큐어 코딩
제출자	[학번 / 이름 입력]
개발 환경	Python 3.11+ · Flask 3.1 · SQLite · pytest
작성일	2026년 7월 21일
저장소	[GitHub 공개 저장소 URL 입력]

요약

본 프로젝트는 요구사항 분석, 시스템 설계, 구현, 보안 점검, 테스트, 유지보수의 전체 개발 주기를 기록하면서 안전한 소형 중고거래 서비스를 구현한다. 회원 인증, 상품 관리와 검색, 1:1 메시지, 신고, 관리자 제재, 가상 포인트 송금을 제공하며 CSRF, 비밀번호 해싱, 권한 검사, 입력 검증, 오류 처리를 핵심 통제로 적용했다.

최종 검증	pytest 15개 통과 · 필수 기능 7개 영역 완료 · 보안 통제 10개 검증
-------	---

1. 프로젝트 개요

본 시스템은 개인 사용자가 중고 상품을 등록하고 다른 사용자와 대화한 뒤 내부 가상 포인트를 이전할 수 있는 교육용 웹 애플리케이션이다. 과제에서 요구한 사용자 가입, 상품 등록·조회, 사용자 소통, 악성 사용자·상품 신고와 관리, 사용자 간 송금, 검색, 관리자 관리 기능을 하나의 Flask 애플리케이션으로 통합했다.

실제 계좌·카드·금융기관 연동은 범위에서 제외하고 송금을 내부 포인트 이전으로 정의했다. 이는 민감 금융정보를 다루지 않으면서 잔액 검증, 중복 처리 방지, 거래 이력 같은 보안 설계를 실습하기 위한 결정이다.

1.1 개발 목표

- 필수 기능을 사용 가능한 웹 화면과 서버 로직으로 완성한다.
- 입력값과 URL 식별자를 신뢰하지 않고 서버에서 다시 검증한다.
- 약점을 발견-원인-수정-검증 형식으로 기록한다.
- pytest와 수동 체크리스트로 결과를 재현 가능하게 만든다.
- GitHub 게시용 실행 문서와 비밀번호 제외 정책을 제공한다.

1.2 범위와 제한

필수 범위에는 텍스트 기반 1:1 메시지와 가상 포인트가 포함된다. 이미지 업로드, WebSocket 실시간 채팅, 실제 결제, 이메일 인증은 제외했다. SQLite는 과제 시연에 적합하지만 고동시성 운영에서는 PostgreSQL과 별도 작업 큐가 필요하다.

2. 요구사항 분석

ID	기능 요구사항	상태
FR-01	회원가입, 로그인, 로그아웃, 프로필 수정	완료
FR-02	상품 등록·목록·상세·수정·삭제	완료
FR-03	상품명·설명 키워드 검색	완료
FR-04	사용자 간 1:1 메시지	완료
FR-05	사용자·상품 신고와 중복 신고 차단	완료
FR-06	가상 포인트 송금과 거래 이력	완료
FR-07	관리자 사용자·상품·신고 관리	완료

2.1 비기능 요구사항

- 비밀번호를 평문 저장하지 않는다.
- 모든 상태 변경 요청은 CSRF 토큰을 요구한다.
- 수정·삭제·관리 기능은 서버에서 권한을 확인한다.
- 가격, 송금액, 길이, 형식, 대상 존재 여부를 검증한다.
- 사용자 입력을 HTML로 실행하지 않는다.
- 오류 화면에 내부 경로, SQL, 스택 추적을 노출하지 않는다.

2.2 요구사항 변경

사용자 간 송금은 플랫폼 내부 가상 포인트 이전으로 구체화했고 실시간 채팅은 요청/응답형 1:1 메시지로 구현했다. 외부 결제와 실시간 인프라의 과도한 복잡성을 줄이고 시큐어 코딩 검증에 집중하기 위한 범위 조정이다.

3. 시스템 설계

3.1 아키텍처

- 브라우저가 Jinja 화면과 WTForms 폼을 사용한다.
- 기능별 Flask Blueprint가 요청을 처리한다.
- Flask-Login과 Flask-WTF가 세션과 CSRF를 담당한다.
- SQLAlchemy ORM이 SQLite에 6개 핵심 모델을 저장한다.
- 공통 오류 처리기가 내부 세부정보를 숨긴다.

3.2 데이터베이스

테이블	핵심 필드	무결성/보안
User	username, password_hash, role, status, balance	아이디 UNIQUE, 해시 저장
Product	seller_id, title, description, price, status	판매자 FK, 가격 검증
Message	sender_id, receiver_id, content	참여자 FK, 길이 제한
Report	reporter_id, target_type, target_id, status	신고 조합 UNIQUE
Transfer	sender_id, receiver_id, amount, idempotency_key	금액 CHECK, 역등기 UNIQUE
AdminLog	admin_id, action, target_type, target_id	관리자 조치 추적

3.3 권한 모델

기능	비회원	일반 사용자	관리자
상품 조회/검색	허용	허용	허용
상품 등록	거부	정상 계정	허용
상품 수정/삭제	거부	본인만	전체
메시지/신고/송금	거부	정상 계정	허용
관리자 화면	거부	403	허용

3.4 주요 화면

화면	URL	설명
홈/상품	/, /products	최근 상품과 검색
인증	/auth/register, /auth/login	계정 생성과 로그인
상품 관리	/products/new, /products/<id>/edit	소유권이 있는 CRUD
메시지	/messages/<user_id>	1:1 대화
신고	/reports/new	사용자·상품 신고
송금	/transfers/new	비밀번호 확인 후 이전
관리자	/admin	사용자·상품·신고 관리

4. 보안 위협 분석과 설계 통제

위협	공격 예	적용 통제
인증정보 유출	DB에서 평문 비밀번호 획득	Werkzeug 단방향 해시
CSRF	삭제·송금 대리 요청	전역 CSRF 토큰
XSS	상품명에 script 저장	Jinja 자동 이스케이프
SQL Injection	검색어로 SQL 조작	SQLAlchemy ORM
IDOR	ID 변경으로 타인 상품 수정	서버 소유권 검사
권한 상승	관리자 URL 직접 호출	역할 데코레이터
금액 변조	음수·초과·자기 송금	범위·잔액·대상 검증
중복 송금	새로고침 재처리	먹등기 UNIQUE
신고 남용	같은 대상 반복 신고	신고 조합 UNIQUE
정보 노출	오류에서 DB 구조 확인	공통 오류 화면

4.1 송금 처리 순서

1. 로그인과 계정 상태를 확인한다.
2. 수신 사용자 존재·정상 상태와 자기 송금 여부를 검사한다.
3. 현재 비밀번호를 해시 비교로 재확인한다.
4. 금액이 1 이상이며 잔액 이내인지 확인한다.
5. 먹등기가 기존 거래에 없는지 확인한다.
6. 두 잔액 변경과 거래 기록을 하나의 커밋으로 반영한다.
7. 무결성 오류 시 전체 세션을 롤백한다.

5. 구현 내용

5.1 인증과 사용자

RegisterForm이 아이디 형식과 비밀번호 길이를 검증하고 서버가 중복 아이디를 다시 확인한다. User.set_password는 해시만 저장한다. 로그인은 존재하지 않는 아이디와 잘못된 비밀번호에 동일한 메시지를 반환하고 정지 계정은 세션을 만들지 않는다.

5.2 상품과 검색

제목 1~100자, 설명 1~2,000자, 가격 1~100,000,000 범위를 강제한다. 수정·삭제는 require_owner를 통과해야 한다. 검색은 ORM의 like를 사용하고 와일드카드를 이스케이프한다.

5.3 메시지와 신고

대화 쿼리는 현재 사용자와 상대방이 참여한 두 방향 메시지만 조회한다. 신고는 대상 존재를 확인하며 DB 고유 제약으로 반복 신고를 방지한다.

5.4 관리자

admin_required가 모든 관리자 라우트에서 역할을 검사한다. 사용자를 정지/활성화하고 상품을 숨김/복구하며 신고를 승인/기각한다. 모든 중요 조치는 AdminLog에 기록한다.

6. 개발 중 보안 약점 수정 기록

ID	발견한 약점	수정	검증
SEC-001	비밀번호 평문 저장 위험	솔트 포함 해시	DB 원문 불일치
SEC-002	타인 상품 IDOR	서버 소유권 검사	타 계정 403
SEC-003	음수·과다 금액	품 범위 + DB CHECK	경계값 거부
SEC-004	중복 송금	역등기 UNIQUE	거래 1건
SEC-005	저장형 XSS	자동 이스케이프	script 비실행
SEC-006	CSRF	전역 토큰	무토큰 400
SEC-007	관리자 권한 상승	역할 검사	일반 사용자 403
SEC-008	반복 신고	복합 UNIQUE	신고 1건
SEC-009	계정 존재 노출	일반화 오류	동일 메시지
SEC-010	내부 오류 노출	공통 오류 화면	세부정보 미노출

상세한 발견-원인-수정-검증 기록은 저장소의 SECURITY_LOG.md에 수록했다.

7. 테스트 계획과 결과

메모리 SQLite와 Flask 테스트 클라이언트를 사용했다. 일반 기능 테스트에서는 CSRF를 비활성화하고, CSRF 전용 테스트에서는 보호를 다시 활성화해 토큰 없는 POST가 400으로 거부되는지 확인했다.

영역	대표 테스트	예상 결과	결과
인증	중복, 해시, 정지 로그인	거부/해시/거부	Pass
상품	CRUD, 검색, 음수 가격	정상/검색/거부	Pass
권한	타인 상품 수정·삭제	HTTP 403	Pass
XSS	script 상품명	HTML 엔티티	Pass
메시지	상대방에게 전송	저장·표시	Pass
신고	동일 상품 2회	1건만 저장	Pass
송금	정상/음수/초과/자기/중복	정상 1건, 나머지 거부	Pass
관리자	일반 접근, 정지, 숨김	403/상태 변경	Pass
CSRF	토큰 없는 POST	HTTP 400	Pass

최종 자동 테스트: 15 passed in 2.77s

8. 유지보수와 개선

8.1 개발 중 개선

- 삭제와 로그아웃을 CSRF가 적용된 POST로 통일했다.
- 송금에 비밀번호 재확인과 먹등키를 추가했다.
- 신고 누적 자동 제재 대신 관리자 검토 흐름을 선택했다.
- 숨김 상품 직접 URL은 권한 없는 사용자에게 404로 응답한다.
- 관리자 조치를 로그로 남긴다.

8.2 향후 개선

- 로그인 속도 제한, 계정 잠금, 이메일 인증을 추가한다.
- 파일 업로드 시 MIME·확장자·크기 검사와 무작위 파일명을 적용한다.
- PostgreSQL 행 잠금으로 동시 송금을 강화한다.
- 메시지 신고, 읽음 상태, 페이지네이션을 추가한다.
- CSP, HSTS 등 운영 보안 헤더와 중앙 로그를 적용한다.

9. 실행 및 제출 방법

1. Python 가상환경을 만들고 `pip install -r requirements.txt`를 실행한다.
2. `.env.example`을 `.env`로 복사하고 `SECRET_KEY`를 변경한다.
3. `flask --app run.py init-db`로 DB를 초기화한다.
4. `flask --app run.py create-admin`으로 관리자 계정을 만든다.
5. `python run.py` 실행 후 `http://127.0.0.1:5000`을 연다.
6. `pytest -q`로 15개 테스트를 실행한다.
7. 비밀과 DB가 제외됐는지 확인한 뒤 GitHub public 저장소에 push한다.

9.1 제출자 정보

표지의 [학번 / 이름 입력]과 [GitHub 공개 저장소 URL 입력]을 실제 정보로 교체한다. 공용 기본 관리자 비밀번호는 코드나 보고서에 기록하지 않고 로컬에서 직접 생성한다.

10. 결론

본 프로젝트는 필수 기능 구현과 함께 각 기능의 신뢰 경계를 서버 측 검증으로 명확히 했다. 비밀번호, 권한, 입력값, 요청 출처, 거래 무결성, 오류 정보를 독립적인 통제로 다뤘고 자동 테스트로 재현 가능한 근거를 남겼다. 과제의 핵심인 개발 과정의 보안 약점 발견과 수정을 요구사항부터 유지보수까지 연결한 완성본이다.

부록 A. 저장소 산출물

파일	목적
README.md	설치, 실행, 관리자 생성, 테스트, 보안 설명
SECURITY_LOG.md	10개 보안 약점 수정 기록
CHECKLIST.md	요구사항 및 보안 테스트 체크리스트
tests/	pytest 기능·보안 테스트 15개
.env.example / .gitignore	환경 변수 예시와 비밀·DB 제외
DEVELOPMENT_REPORT.pdf	개발 전 과정 보고서